

Modellrakterter

Aktiveter for Fysikken i Fokus

Andøya Space Education

2025



Beskrivelse

Dette aktivitetsheftet inneholder byggeinstruksjoner, oppgaver og teori knyttet til rakettforsøket som skal gjennomføres under kurset. Aktivitetene er tilpasset elever på videregående nivå med fysikk som programfag, men kan fint brukes av andre som vil lære mer om hvordan raketter fungerer. Aktivitetene er utviklet av Andøya Space Education som en del av prosjektet Fysikken i Fokus, og inkluderer blant annet; bygging av modellraketter, bygging av nyttelast, simuleringer, baneberegninger, utvikling av egne forsøk og vurdering av sikkerheten ved rakettoppskytinger. Ved å gjennomføre aktivitetene vil deltakerne få innsikt i det teoretiske, og det praktiske arbeidet som er nødvendig for at en rakettoppskyting skal skje, og gi erfaring med hvordan fysikkfaget brukes i praksis.

Mål for kurset

- Utforske hvordan fysikk kan brukes i praksis til å bestemme egenskapene til en rakett
- Bli kjent med viktige prinsipper knyttet til rakettstabilitet
- Lære hvordan man konstruerer og skyter opp en rakett
- Reflektere over hvorfor man skyter opp raketter
- Erfare hvordan numerisk metode kan brukes til å lage modeller for banen til en rakett
- Vurdere sikkerheten rundt en rakettoppskyting

Relevante kompetansemål (FYS01-02)

- Planlegge, gjennomføre og videreutvikle forsøk, og analysere data og beregne usikkerhet for å vurdere gyldigheten av funn
- Bruke numeriske metoder og programmering til å utforske og modellere fysiske fenomener
- Utforske, beskrive og modellere bevegelse i to dimensjoner
- Gjøre rede for hvordan krefter kan forårsake krumlinjet bevegelse, og bruke dette i beregninger
- Utforske og analysere en selvvalgt teoretisk eller praktisk problemstilling i fysikk, og presentere viktige prinsipper, sammenhenger og konsekvenser
- **Tverrfaglige tema:** Demokrati og medborgerskap

Tid

8–10 timer

Utstyr

- Skrivesaker
- Kalkulator
- PC
- Byggesett: rakett med tilhørende verktøy
- Byggesett: nyttelast til modellrakett

Innhold

Rakettfysikk	4
Modeller og baneberegninger	4
Stabilisering av raketter	6
Rakettmotorer	8
Formelsamling	9
Aktivitet 1: Statisk test av rakettmotor	10
Aktivitet 2: Numerisk baneberegning	12
Oppgave 1: Krefter på raketten	12
Oppgave 2: Numerisk baneberegning med Python	13
Oppgave 3: Gjør baneberegningene mer presis	13
Aktivitet 3: Baneberegninger og sikkerhet	14
Oppgave 1: Bygging og sikkerhet	14
Oppgave 2: Motor- og rakettkarakteristikk	14
Oppgave 3: Baneberegninger	14
Oppgave 4: Estimering av luftmotstand	15
Oppgave 5: Rakettligningen	15
Aktivitet 4: Simulering med OpenRocket	16
Aktivitet 5: Databehandling	17
Oppgave 1: Plott og les av data med Python	17
Oppgave 2: Dataanalyse	17
Instruksjoner: Bygging av modellrakett	18
Instruksjoner: Nyttelast til modellrakett	20
Oppgave 1: Last opp kode til nyttelasten	21
Oppgave 2: Test	21
Oppgave 3: Montering	22
Presentasjoner	23
Referanser	24
Vedlegg	25
Vedlegg 1: Nedtellingsprosedyre	25
Vedlegg 2: Rakettmotorspesifikasjoner (Klima)	29

Rakettfysikk

Modeller og baneberegninger

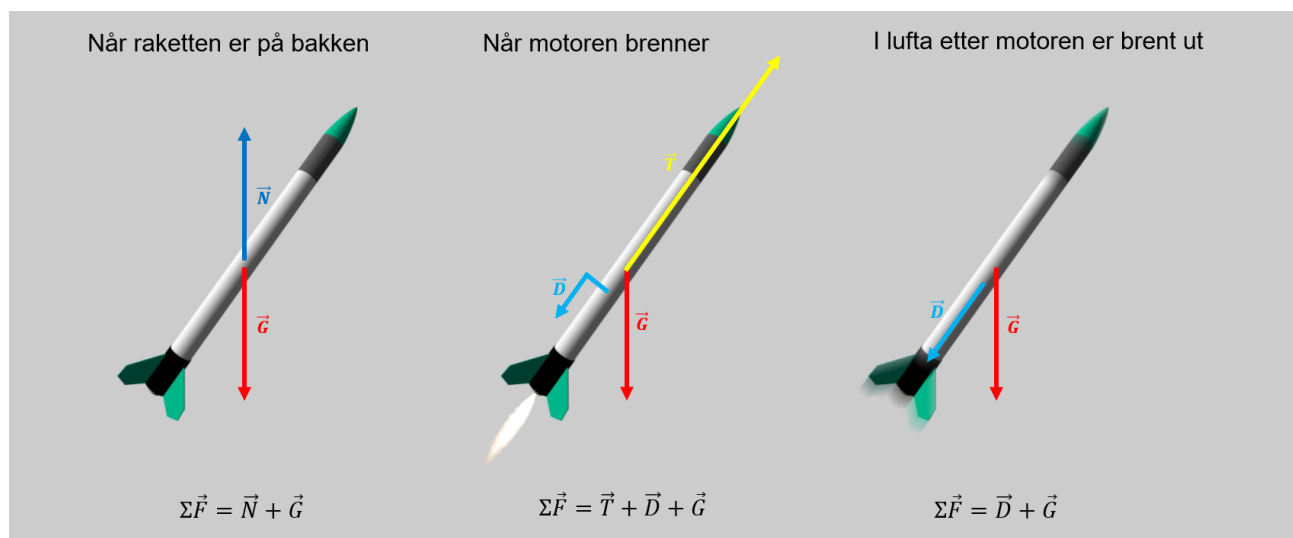
Baneberegninger er viktige for å forstå hvordan raketter beveger seg. Grunnlaget for baneberegningen i dette oppgavesettet er Newtons andre lov, $\sum \vec{F} = m\vec{a}$. Summen av kreftene som virker på raketten er lik massen multiplisert med akselerasjonen. Vi kan altså regne ut akselerasjonen hvis vi kjenner kreftene. Og når akselerasjonen er kjent kan banen beregnes ved å bruke bevegelsesligningene som beskriver posisjon og hastighet over tid.

$$\text{Posisjon: } \vec{r}(t) = \frac{1}{2}\vec{a}t^2 + \vec{v}_0t + \vec{r}_0$$

$$\text{Hastighet: } \vec{v}(t) = \vec{a}t + \vec{v}_0$$

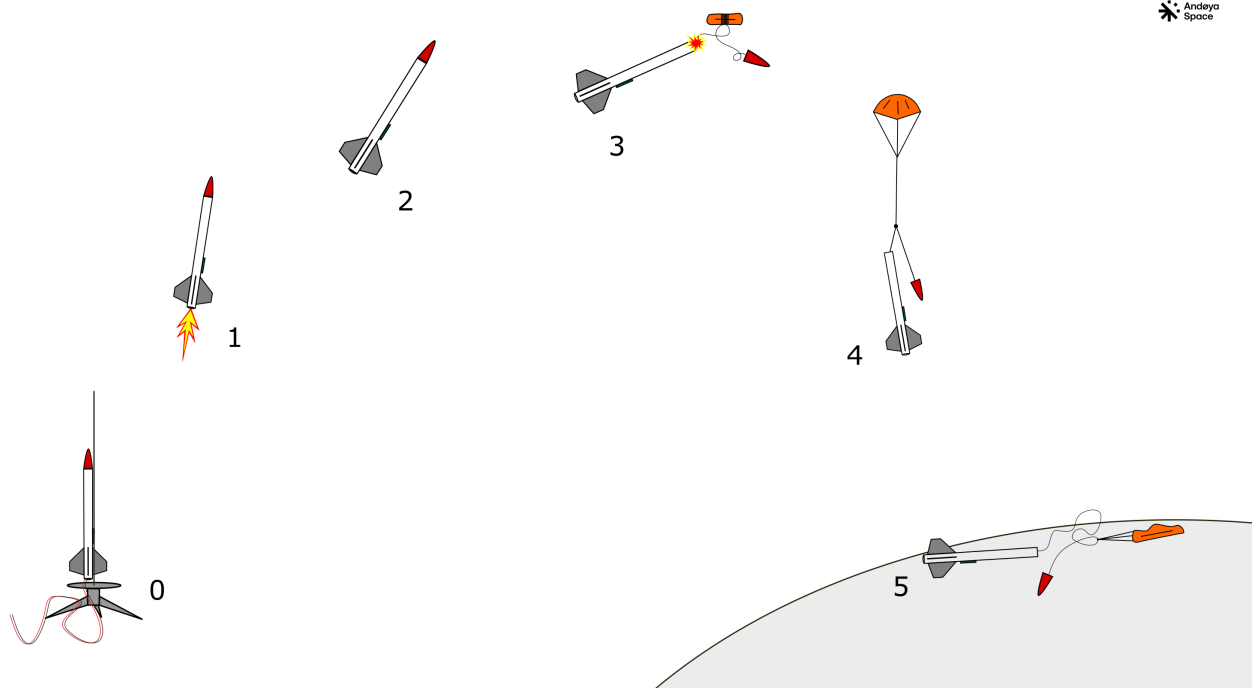
Aktivitetene i dette heftet vil gi veiledning i hvordan bevegelsesligningene for raketter kan løses både ved programmering og med penn og papir.

Men før bevegelsesligningene kan løses må altså akselerasjonen, og dermed kreftene som virker på raketten være kjent – se figur 1. En kraft, la oss si tyngdekraften, beskrives ved å bruke en modell. En modell er en en forenklet kopi av noe mer komplekst. $\vec{G} = m\vec{g}$ som ofte blir brukt til å beskrive tyngdekraften, fungerer til vår hensikt som er å gjøre baneberegninger på raketter like over jordas overflate. Men i virkeligheten har denne modellen en stor begrensning. Tyngdekraften vil avta etterhvert som avstanden til jorda øker, altså jo høyere raketten flyr. For raketter som går veldig høyt er det nødvendig å bruke en mer nøyaktig modell, hvor tyngdekraften avtar med høyden, $\vec{G}(r)$.



Figur 1: Kreftene som virker på raketten ved 3 situasjoner

Hendelsesforløpet til raketten når den er i lufta, som vist i figur 2, er en viktig modell som blir brukt til å forutse, og gjøre beregninger på raketts bane. Hendelsene beskrevet i modellen er de vi antar vil inntreffe etter rakettmotoren er antent. For hver hendelse i figur 2 kan vi lage modeller som beskriver kreftene på raketten, og deretter gjøre baneberegninger basert på disse. Baneberegninger gjør det mulig å spå fremtiden - vi kan forutse hvordan bane raketten omentrent kommer til følge når den skytes opp, basert på det vi vet om raketten og omgivelsene.



Figur 2: Hendelsesforløpet til raketten etter oppskyting

I lista under beskriver t_b brenntiden til rakettmotoren og t_d glidetiden, eller tiden det tar fra rakettmotoren slutter å brenne til fallskjemen skytes ut.

0. Raketten står i ro på rampen:
Tid = 0, $\sum \vec{F} = 0$, hastighet = 0, posisjon = 0
1. Rakettmotoren brenner:
Tid = t_1 , $\sum \vec{F} = \vec{T} - \vec{D} - \vec{G}$, hastighet = $\vec{v}(t)$, posisjon = $\vec{p}(t)$
2. Rakettmotoren har akkurat har sluttet å brenne:
Tid = t_b , $\sum \vec{F} = -\vec{D} - \vec{G}$, hastighet = $\vec{v}_{\max}$, posisjon = $\vec{p}(t_b)$
3. Fallskjemen kommer ut:
Tid = $t_b + t_d$, $\sum \vec{F} = -\vec{D} - \vec{G}$, hastighet = $\vec{v}(t_b + t_d)$, posisjon = $\vec{p}(t_b + t_d)$
4. Raketten daler ned med fallskjem:
Tid = t_4 , $\sum \vec{F} = -\vec{D}_{fallskjerm} - \vec{G}$, hastighet = $\vec{v}(t_4)$, posisjon = $\vec{p}(t_4)$
5. Raketten har landet på bakken igjen:
Tid = t_5 , $\sum \vec{F} = 0$, hastighet = 0, posisjon = 0

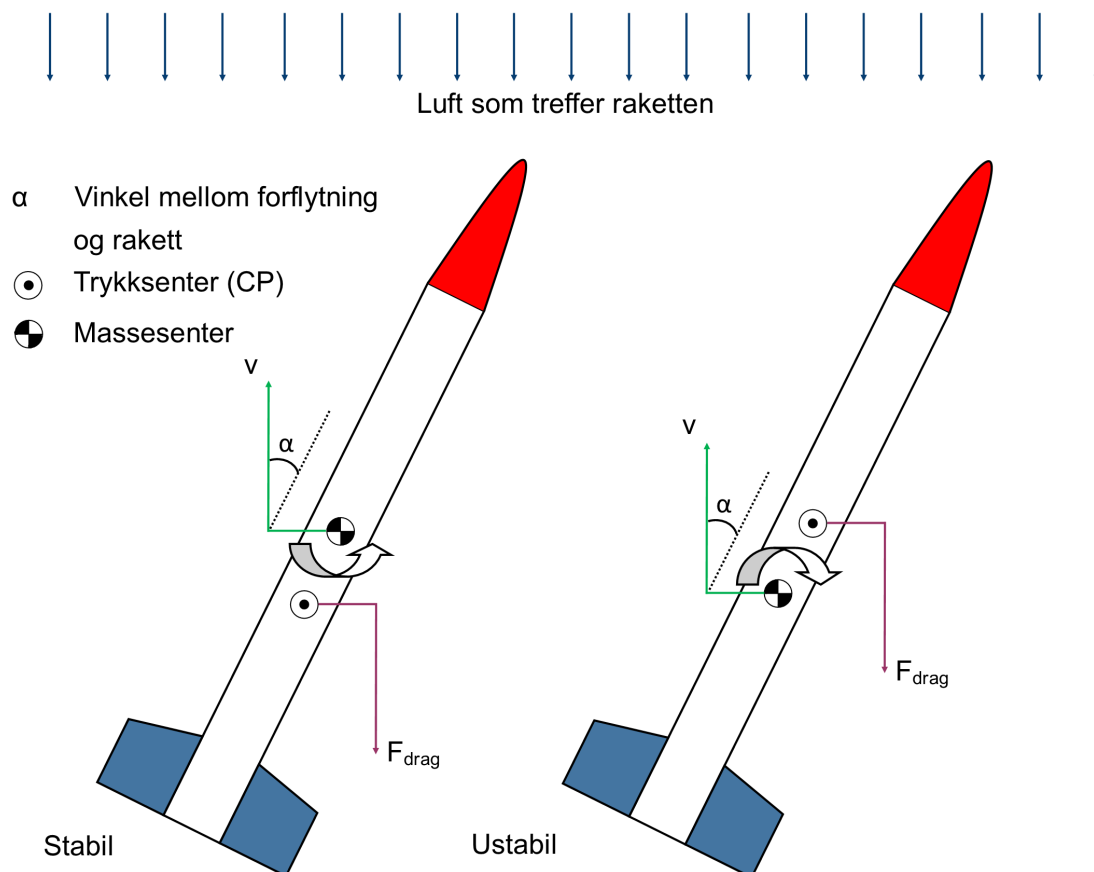
Stabilisering av raketter

Enhver rakett som skal skytes opp trygt, må ha en form for stabilisering. En stabil rakett vil si at den beveger seg i en relativt rett linje. Denne egenskapen oppnås ved at raketten selv er i stand til å motstå endringer i bevegelsen [1]. En ustabil rakett vil avvike fra sin opprinnelige retning og mest sannsynlig krasje i bakken, noe som gjør den farlig å skyte opp.

For store raketter som skytes opp brukes gjerne avanserte systemer som thrust vectoring og spinnstabilisering for å holde riktig kurs [2], mens for modellraketter som skytes opp i lav høyde er det vanligere å bruke en teknikk som kalles aerodynamisk stabilisering. For å finne ut om en rakett er aerodynamisk stabil må vi kjenne til angrepspunktet til tyngdekraften, og angrepspunktet til luftmotstanden på raketten. Angrepspunktet, eller resultantkraften er der vi antar summen av en bestemt kraft virker på et objekt. Tyngdekraften virker i det vi kaller massesenteret, fra nå av kalt CG (center of gravity) og luftmotstanden virker i det vi kaller for trykksenteret, fra nå av kalt CP (center of pressure). Forholdet mellom plasseringen til disse to punktene avgjør om raketten er stabil eller ikke. Enkelt forklart må CG på raketten være foran CP. Ideelt med en stabilitetsmargin mellom 1 og 2 [1].

$$\text{Stabilitetsmargin} = \frac{CP - CG}{d_r}$$

I formelen over er d_r diameteren til raketten, CG er posisjonen til massesenteret, og CP posisjonen til trykksenteret relativt til toppen av nesekonen. Men hvorfor er det egentlig slik?

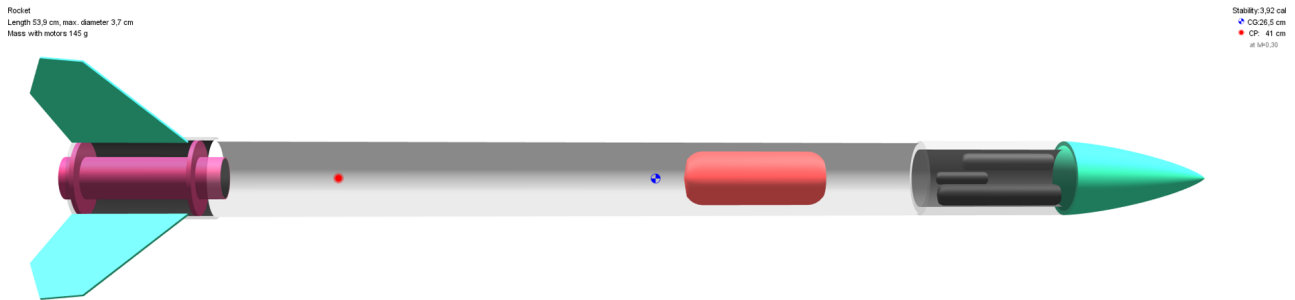


Figur 3: Raketter vil rotere rundt sitt eget massesenter

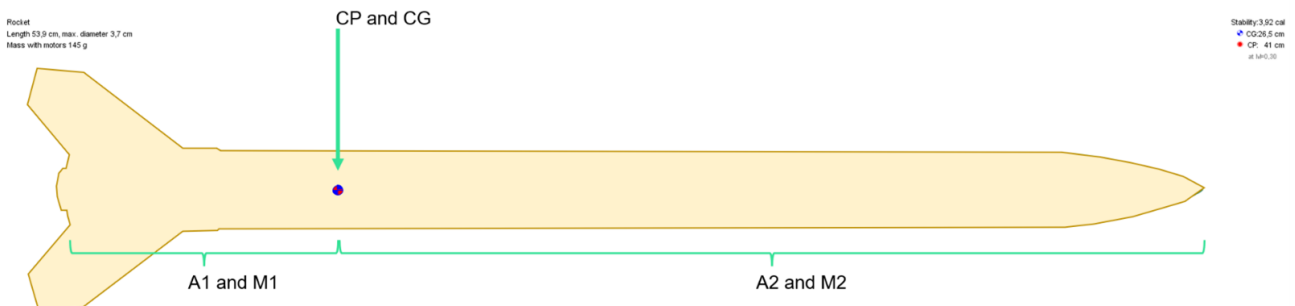
Se for deg at en rakett flyker mot himmelen med stor hastighet, men som av ukjente grunner får en liten dreining i forhold til retningen den forflytter seg i, slik som i figur 3. I tilfellet hvor CG er plassert foran CP vil raketten dreie inn i forflyttningsretningen og stabilisere seg selv. Raketten er altså stabil. I tilfellet hvor CP er foran CG, vil luftmotstanden dreie raketten rundt CG noe som resulterer i en ustabil rakett.

CG og CP

CG er det punktet på raketten der massen er fordelt likt på begge sider, som vist i figur 4. For å finne CG kan en linjal eller et annet smalt objekt brukes for å balansere raketten. På det punktet hvor raketten er i balanse, vil CG befinne seg.



Figur 4: Raketten CP og CG, beregnet med OpenRocket



Figur 5: 2D projeksjonens CP og CG

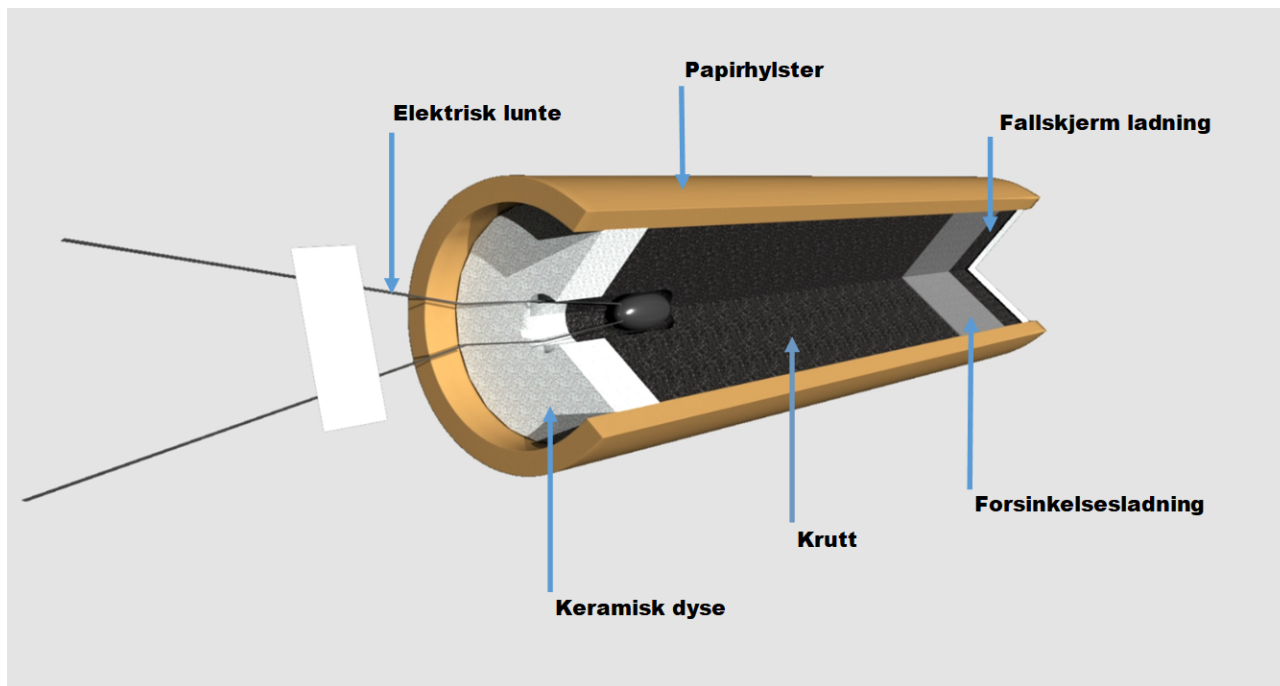
CP er det punktet på raketten hvor alle de aerodynamiske kreftene er i likevekt. Å finne posisjonen til CP er en relativt komplisert prosess som krever mye matematikk. Imidlertid finnes det en enklere metode for å estimere posisjonen til CP for en modellraket. Ved å anta at CP befinner seg i det punktet hvor overflatearealet til raketten er likt på alle sider, kan vi bruke følgende metode:

1. start med å lage en flat skisse av tverrsnittet til raketten på en papplate, slik som vist i figur 5
2. klipp deretter ut den flate projeksjonen
3. til slutt måles posisjonen til massesenteret på den flate projeksjonen

Siden massen til papp er jevnt fordelt på innsiden, bestemmes posisjonen til massesenteret av arealets fordeling på hver side. Dette punktet er omtrent CP [1].

Rakettmotorer

Modellrakettmotorer er små faststoffmotorer som skal gi raketten skyvekraften den trenger for å fly. Motoren skal også sørge for at fallskjermen kommer ut på riktig tidspunkt. Drivstoffet i en modellrakettmotor består av brennstoff og et oksidasjonsmiddel blandet sammen til en fast masse. Som vist i figur 6 består motoren av et papirhylster, en dyse og ladningen som skal skyte ut fallskjermen i tillegg til selve drivstoffet [3].



Figur 6: Skjematisk tegning av rakettmotorens innside. Illustrasjon: *apogeerockets*

Det finnes forskjellige kategorier av modellrakettmotorer. Kategoriene kjennetegnes av en kode bestående av én bokstav og to tall. Bokstaven forteller oss hvor mye impuls motoren leverer. Total impuls er produktet av skyvekraft (thrust) og tiden motoren produserer skyvekraft, også kalt brenntid. Figur 7 under viser inndelingen av de vanligste motorklassene. Det første tallet i koden angir gjennomsnittlig skyvekraft i newton, og det siste tallet angir glidetiden som angir tiden det tar fra motoren har sluttet å brenne til fallskjermen skytes ut.

Motorklassifisering	Klasse	Total impuls (newtonsekund)
F.eks. C6-3	1/2A	0,625 - 1,25
↑ Tid mellom brennslutt og forsinkelsesladning (s)	A	1,26 - 2,50
↑ Gjennomsnittlig skyvekraft (N)	B	2,51 - 5,00
↑ Totalimpuls klasse	C	5,01 - 10,00
	D	10,01 - 20,00

Figur 7: Motorkoder og klassifiseringssystem for rakettmotorer

Formlelsamling

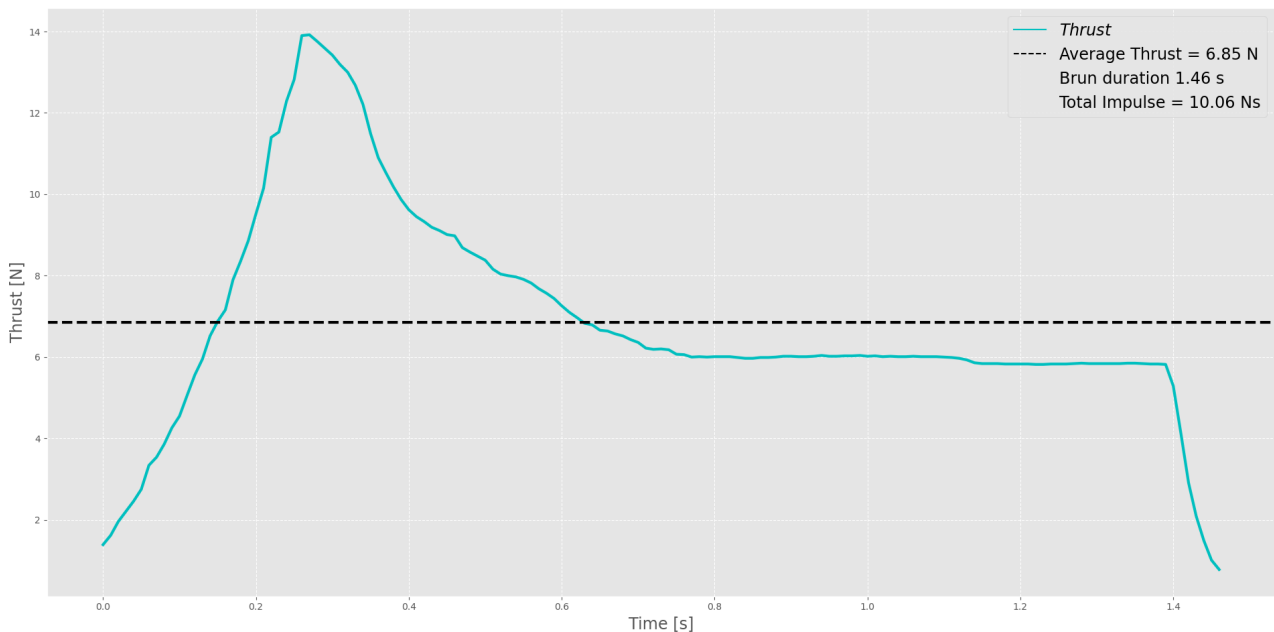
Totalimpuls	$I_t = \int T(t) \, dt = T_{\text{ave}} t_b$
Drivstofforbruk	$q_p = \frac{m_p}{t_b}$
Eksoshastighet	$v_e = \frac{I_t}{m_p} = \frac{F_{\text{ave}}}{q_p}$
Spesifikk impuls	$I_{\text{sp}} = \frac{v_e}{g_0}$
Newtons 2. lov	$\sum \vec{F} = m\vec{a}$
Bevegelsesligning 1 (hastighet)	$\vec{v}'(t) = \vec{a}(t) \iff \vec{v}(t) = \int \vec{a}(t) \, dt$
antar konstant akselerasjon	$\vec{v}(t) = \vec{a}t + \vec{v}_0$
Bevegelsesligning 2 (posisjon)	$\vec{r}'(t) = \vec{v}(t) \iff \vec{r}(t) = \int \vec{v}(t) \, dt$
antar konstant akselerasjon	$\vec{r}(t) = \frac{1}{2}\vec{a}t^2 + \vec{v}_0t + \vec{r}_0$
Rakettligningen	$\Delta v = v_e \ln \left(\frac{m_0}{m_1} \right)$
Luftmotstandsfaktor	$\beta = \frac{1}{2}C_D\rho A_{\text{max}}$
Luftmotstand	$D = \vec{D} = \beta \vec{v} ^2$
Luftmotstand dekomponert	$D_x = -\beta v_x \vec{v} \quad \text{og} \quad D_y = -\beta v_y \vec{v} $

Aktivitet 1: Statisk test av rakettmotor

For å bli bedre kjent med virkemåten til rakettmotoren og hvilke spesifikasjoner den har kan det gjennomføres det en statisk motortest. I en statisk motortest fyres motoren på en fastmontert testbenk. Testing av utdelt rakettmotor gjennomføres med PASCO kraftmåler, og data samles inn med en PASCO datalogger. Målingen må gjøres ved oppsatt stasjon for motortesting. Målet med testingen er å måle gjennomsnittlig skyvekraft, brenntid og totalimpuls for motoren som skal brukes i modellraketten slik som vist i figur 8. Det er også mulig å eksportere datasett (via Bluetooth) for skyvekraft over tid. Datasettet kan brukes til å lage mer realistiske simuleringer av rakettskuddet.

Utstyr

- PASCO datalogger
- Trådløs PASCO kraft- og akselerasjonssensor
- Testbenk
- Bruk kun **pluggede!!!** rakettmotorer. Kompatible motorer: B2-P, B4-P, C2-P, C6-P, D3-P og D9-P
- Tenner og lunte
- Vernebriller



Figur 8: Thrustcurve for C6 motor - *filter_Thrustcurve.py*

Forberedelser

1. Bruk vernebriller for å beskytte øynene under motortesten
2. Monter motoren i testbenken. Sjekk at kraftsensoren er slått på. Gummiknappen på kraftsensoren skal være i kontakt med motoren. Motoren skal være rettet med dysa vend utover
3. Koble PASCO dataloggeren til sensoren via Bluetooth. Bruk *Thrustcurve*-oppsettet på dataloggeren, eller opprett et nytt eksperiment som plotter kraft over tid. For å teste sensoren, trykk på startknappen som er på dataloggeren. Hvis et plott vises, fungerer oppsettet korrekt. Hvis ikke, gjenta steg 2 og 3
4. Sjekk at tenneren er desarmert. Koble tenneren til lunta, og sett deretter lunta inn i rakettmotoren. Sørg for at ingen oppholder seg bak motoren. Alle tilskuere skal ha en minimum avstand på 5 meter ut til sidene fra motoren før testen kan gjennomføres
5. Start dataloggingen, start nedtelling fra 5 og fyr av motoren. Stopp datalogging etter at motoren er utbrent
6. Testen er fullført. Gjenta prosessen for å gjennomføre flere tester.

Datainnsamling og analyse

Alternativ 1:

Eksporter data fra PASCO til en telefon via Bluetooth, og overfør deretter datasettet til en datamaskin med Python IDE installert, f.eks. Spyder. Bruk *filter_Thrustcurve.py*, eller skriv egen kode for å analysere datasettet. Målet er å få plottet skyvekraft over tid, beregne gjennomsnittelig skyvekraft og estimere motorens brenntid.

Alternativ 2:

Bruk innebygde analysefunksjonen på PASCO dataloggeren til å måle og beregne brenntid og total impuls for motoren. Beregn gjennomsnittelig skyvekraft.

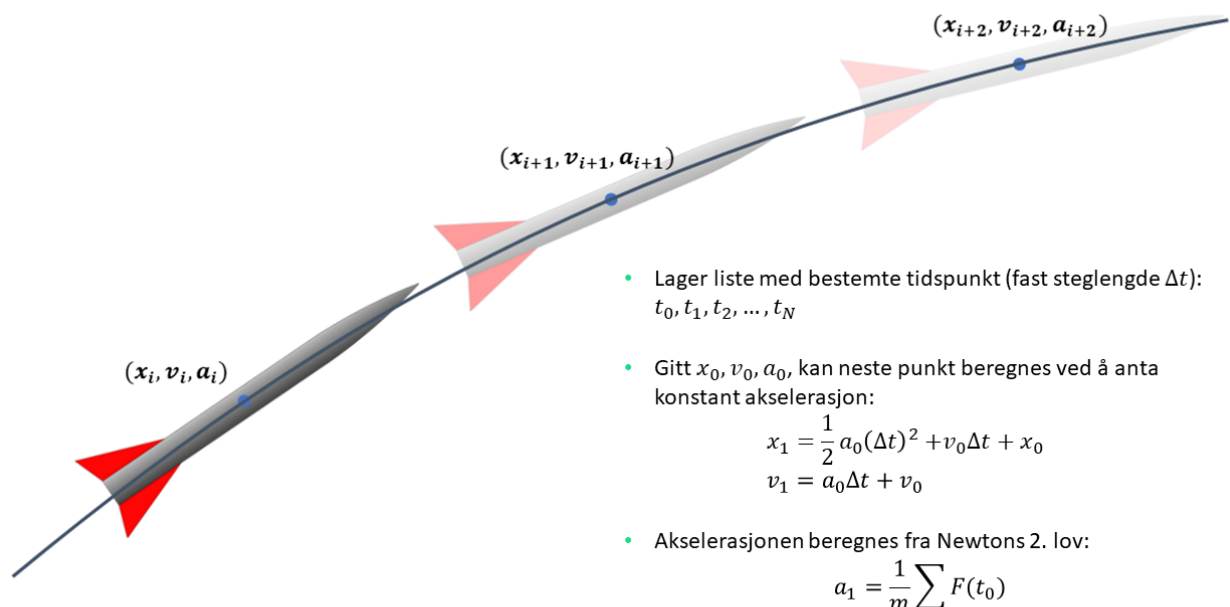
Aktivitet 2: Numerisk baneberegning

Numerisk metode brukes for å lage modeller som kan forutsi hvordan en rakett vil fly. En slik modell benytter en liste med faste tidspunkter hvor raketts akselerasjon, hastighet og posisjon skal beregnes. Tiden mellom hvert punkt i lista kalles Δt . Vi antar at raketts akselerasjon er konstant mellom hver Δt . Denne antagelsen gjør det mulig å bruke bevegelsesligningene til å oppdatere raketts posisjon og hastighet ved hvert tidspunkt.

$$\text{Posisjon: } \vec{r}(t) = \frac{1}{2} \vec{a} t^2 + \vec{v}_0 t + \vec{r}_0$$

$$\text{Hastighet: } \vec{v}(t) = \vec{a} t + \vec{v}_0$$

Men før den numeriske baneberegningen starter må vi definere startbetingelsene, altså start akselerasjon, hastighet og posisjon. Først når vi har dette kan endringen i raketts hastighet og posisjon beregnes. Ved å gjøre tiden Δt mellom hvert punkt veldig liten, kan vi øke nøyaktigheten i de numeriske beregningene. Denne metoden kalles Eulers metode [4].



Figur 9: Ved å anta konstant akselerasjon mellom x_i, x_{i+1} og x_{i+2} kan vi estimere hastighet og posisjon når vi vet startbetingelsene

For raketten i figur 9 kan vi for eksempel finne hvor langt raketten vil fly etter $i\Delta t, (i+1)\Delta t$ og $(i+2)\Delta t$ sekunder forutsatt at vi vet startposisjon, massen og hvilke krefter som påvirker den. For å estimere akselerasjon i neste punkt, antar vi at akselerasjonen er konstant mellom dette punktet og forrige. Deretter oppdateres verdiene for hvert punkt. Et eksempel på implementering i Python finnes neste side.

Oppgave 1: Krefter på raketten

Redegjør for kreftene som virker på raketten i følgende tilfeller:

- Når motoren brenner
- Under glideflukten (etter motoren er brent ut)

Oppgave 2: Numerisk baneberegning med Python

I de neste oppgavene kan det utdelte rammeverket for numeriske baneberegninger (*NumeriskRakett_start.py*) brukes, eller så kan en ny kode skrives fra bunn

- Skriv inn nødvendige konstanter for simuleringen: utskytingsparametre, rakettmotor, vekt og aerodynamikk
- Bruk Newtons 2. lov for å beskrive akselerasjonen til raketten i while-løkkene. Akselerasjonen skal beregnes i x- og y-retning. Det er derfor nødvendig å dekomponere kreftene. Se formelsamlingen for beskrivelse av luftmotstanden
- Sett opp uttrykk for hastighet og posisjon i while-løkkene. Bruk bevegelsesligningene for hastighet og posisjon som er beskrevet i formelsamlingen
- Plott banen til raketten og lagre plottene slik at de kan vises under presentasjonen
- Test koden med forskjellige tidsintervaller, Δt . Kommenter gjerne resultatet

Oppgave 3: Gjør baneberegningene mer presis

De siste oppgavene i denne aktiviteten krever litt programmering for å kunne løses

- Bruk tidsavhengig masse i stedet for konstant masse. Vi kan anta konstant drivstofforbruk under hele brenntiden
- Simuler banen med tidsavhengig skyvekraft ved å bruke data fra statisk motortest

```
# Eksempel: Simulering av rakettbane med konstant akselerasjon og uten luftmotstand
# Importere biblioteker
import matplotlib.pyplot as plt
import numpy as np

# Startbetingelser
a_trust = 40 # Akselerasjonen fra motoren (skyvekraft/masse) i m/s^2
g = 9.81 # Akselerasjon fra tyngdekraften i m/s^2
a = a_trust - g # Total akselerasjon mens motoren brenner
tb = 1 # Brenntid i sekund
dt = 0.01 # Tidssteg i sekund
time = np.arange(0, 10, dt) # Liste med tidspunkter fra 0 til 10 sekund
i = 0 # Heltallstilleren "i" starter med verdien 0
i_max = np.size(time) - 1 # Setter "i_max" lik antall steg i lista med tidspunkter
v = np.zeros(len(time)) # Liste for lagring av hastighetsverdier. Starthastighet = 0 m/s
r = np.zeros(len(time)) # Liste for lagring av posisjonsverdier. Startposisjon = 0 m

# Beregn banen
while time[i] < tb:
    v[i+1] = a*dt + v[i] # Beregn hastighet i m/s
    r[i+1] = 1/2*a*dt**2 + v[i]*dt + r[i] # Beregn posisjon i m
    i += 1 # Heltallstiller, i + 1
while i < i_max and r[i] >= 0: # Stopp ved i = i_max,
    # eller hvis raketten treffer bakken (r = 0)
    a = -g # Total akselerasjon etter motoren er stoppet
    v[i+1] = a*dt + v[i] # Hastighet i m/s
    r[i+1] = 1/2*a*dt**2 + v[i]*dt + r[i] # Posisjon i m
    i += 1

# Plot banen
plt.plot(time, v, label = 'Hastighet i m/s')
plt.plot(time, r, label = 'Posisjon i m')
plt.xlabel('Tid (s)')
plt.ylabel('Hastighet/Posisjon')
plt.title('Numerisk baneberegning, eksempel uten luftmotstand')
plt.legend()
plt.show()
```

Aktivitet 3: Baneberegninger og sikkerhet

Oppgave 1: Bygging og sikkerhet

- Bygg en modellrakett. Se **instruksjoner: Bygging av modellrakett**
- Mål og noter ned følgende:
 - Masse [kg] med og uten motor
 - Motortype [f.eks. A6-4]
 - Raketts diameter [m] og største frontareal inkludert finnene
 - Lengde fra toppen av nesekonen til CG [m]. OpenRocket kan brukes her
 - Lengde fra toppen av nesekonen til CP [m]. OpenRocket kan brukes her
- Basert på informasjonen vi har, er raketten stabil?
- Gå nøye igjennom nedtellingsprosedyren, se **vedlegg 1: Nedtellingsprosedyre**. Fordel roller og fyll ut det som kan fylles ut i skjemaet.

Oppgave 2: Motor- og rakettkarakteristikk

Fyll ut tabellen under. Se **vedlegg 2: Rakettmotorspesifikasjoner (Klima)** for detaljert informasjon om motoren

Navn på raketten:	Motortype:		
Spesifikasjon	Symbol	Fyll inn	Enhet
Maks løftevekt	m_{maks}		kg
Totalvekt på ferdig rakett inkl. motor	m_{tot}		kg
Drivstoffvekt	m_d		kg
Totalimpuls	I_t		Ns
Gjennomsnittlig skyvekraft	T_{ave}		N
Brenntid	t_b		s
Glidetid	t_d		s
Gjennomsnittlig drivstofforbruk	q_p		kg/s
Eksoshastighet	v_e		m/s
Spesifikk impuls	I_{sp}		s

Oppgave 3: Baneberegninger

Bruk Newtons 2. lov og bevegelsesligningene til å finne svarene i de neste oppgavene. Husk at massen til raketten endrer seg når rakettmotoren brenner. Bruk derfor raketts gjennomsnittlige masse, $m = (m_{\text{start}} + m_{\text{slutt}})/2$.

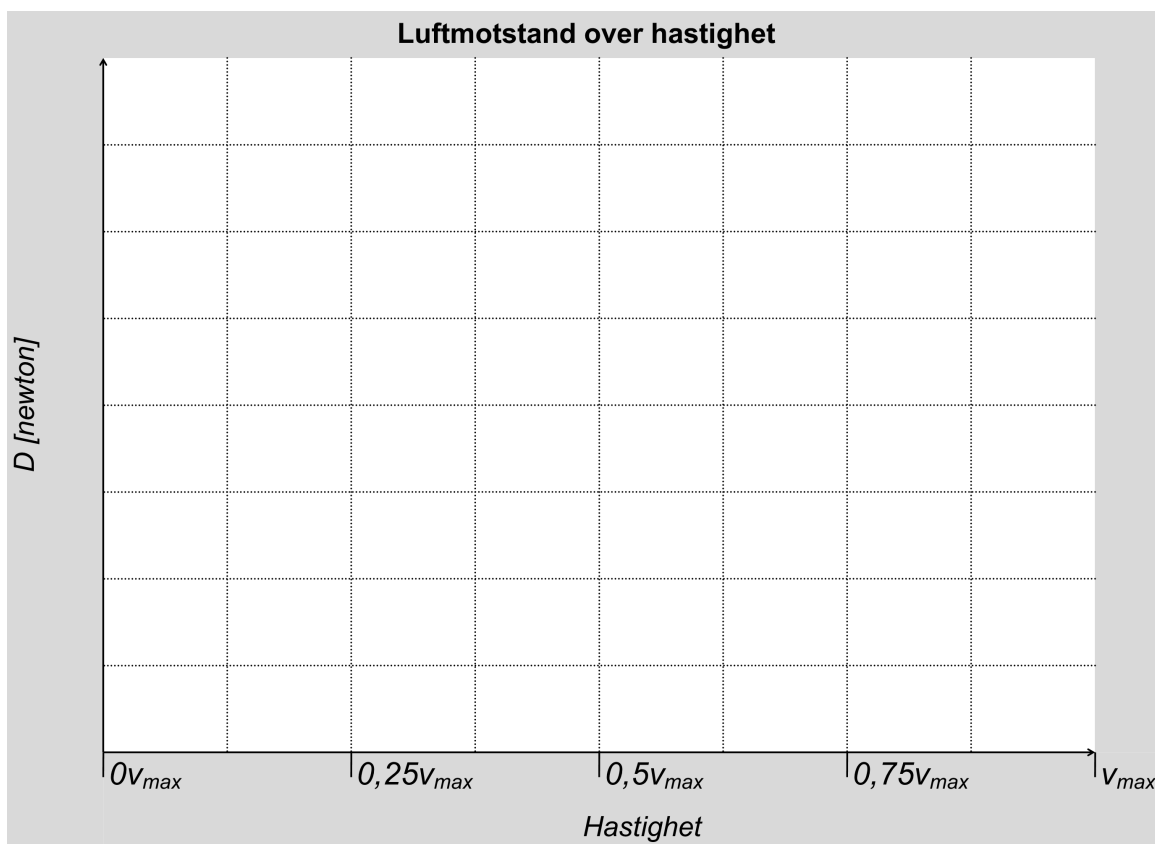
- Regn ut raketts ideelle brennslutthastighet, v_{max} . Dvs. i en situasjon der raketten ikke er påvirket av ytre krefter, dvs. gravitasjon og luftmotstand.
- Anta nå at raketten påvirkes av gravitasjon, og **ikke** luftmotstand. Regn ut brennslutthastighet, v_{max} . Hvis ikke elevasjon (vinkel på horisontalplan) er kjent bruk 80°
- Regn ut brennslutthøyden, h_b
- Regn ut tiden det tar fra rakettmotoren slutter å brenne til raketten når sitt høyeste punkt h_{max} , også kalt apogee
- Regn ut høyden ved apogee, h_{max} .

Oppgave 4: Estimering av luftmotstand

- a) Mål raketts største frontareal, $A_{\max}$, og estimer raketts luftmotstandsfaktor, β . Anta at lufttettheten $\rho = 1,14 \text{ kg/m}^3$, og luftmotstandskoeffisienten $C_D = 0,8$

Anta at $v_{\max}$ fra oppgave 3b) er den teoretisk høyeste hastighet som raketten kan oppnå. Bruk denne hastigheten til å estimere luftmotstand i neste oppgave

- b) Bruk formelen for luftmotstand og plott luftmotstand i figuren under. Følgende hastigheter skal brukes: $0, 0,25v_{\max}, 0,5v_{\max}, 0,75v_{\max}$ og $v_{\max}$. Diskuter om resultatet er realistisk. Kommenter gjerne!
- c) Regn ut raketts terminalhastighet v_t og marker hvor denne inntreffer i grafen under.
Hint: terminalhastighet oppnås når luftmotstand og tyngdeakselerasjon utligner hverandre.



Oppgave 5: Rakettligningen

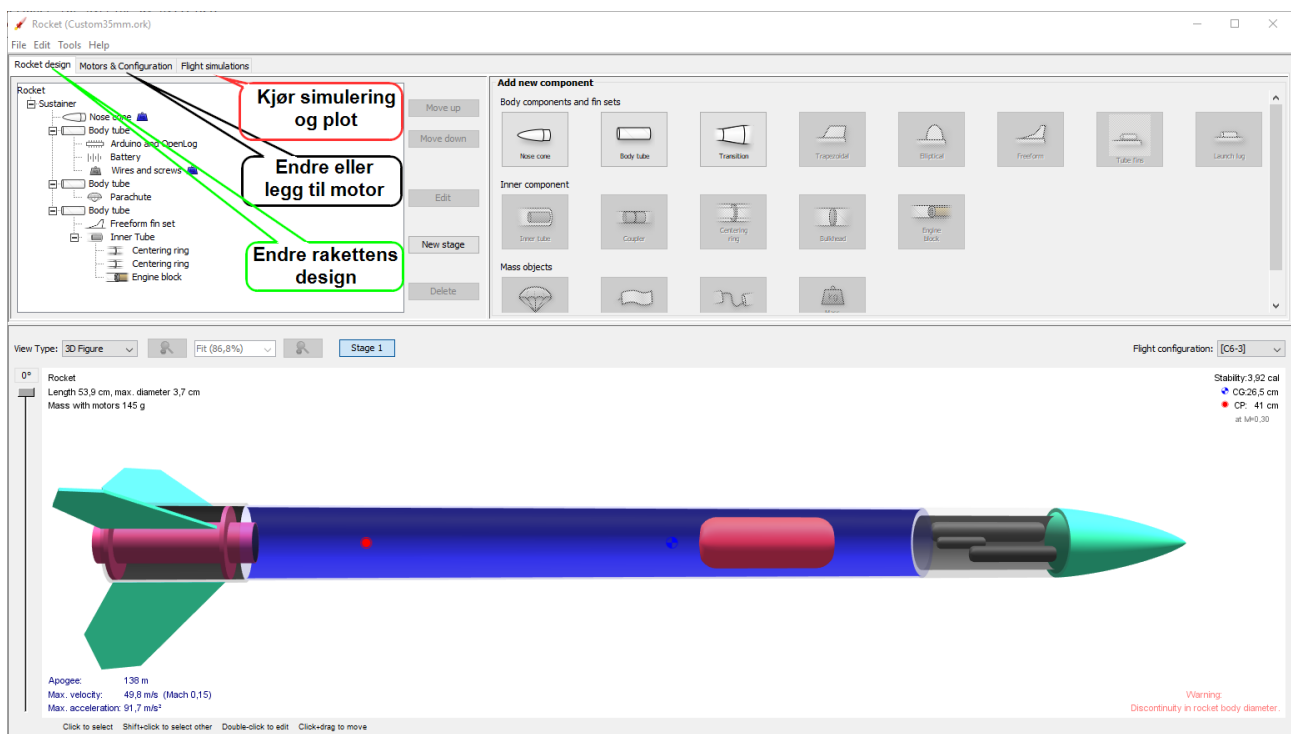
Bruk rakettligningen (se formelsamlingen) til å regne ut $v_{\max}$. Sammenlign resultatet med svaret fra oppgave 3b) og kommenter resultatet.

Aktivitet 4: Simulering med OpenRocket

OpenRocket er et simuleringsverktøy for raketter som er gratis, og enkelt å bruke. Programvaren gir mulighet til å tegne og designe egne raketter, samt redigere i en rekke variabler før simuleringen gjennomføres. OpenRocket kan lastes ned her: openrocket.info/

- Åpne OpenRocket og last inn **.ork** filen for rakett-designen din.
Velg **design** → **File** → **Open** for å åpne filen. Spør en av instruktørene hvis du ikke vet hvor du finner **.ork** filen
- Finn riktig motor for din rakett.
Velg **Motors & Configuration** → **New Configuration** → **Select motor**
 - Finn produsenten av motoren, for eksempel Klima
 - Velg riktig type motor, for eksempel A6
 - Velg riktig tid for forsinkelsesladningen, for eksempel 4 sekunder
- Noter stabilitetsmargin (stability), CG og CP. Sjekk om raketten er stabil. Hvis ikke, diskuter hva som kan være årsaken.
- Velg **Flight simulations** → **New simulation** → **Simulate & Plot**
Legg inn vindstyrke fra værmelding og en realistisk vinkel (elevasjon) på raketttrampen (som vanligvis er ca. 80°). Dette kan justeres under **Edit simulation**. Plott høyde, vertikal akselerasjon og vertikal hastighet over tid. Avgjør basert på plottene tiden når rakettmotoren brente, når raketten var i fritt fall og når fallskjermen kom ut. Lagre plottet. Det kan inngå som en del av presentasjonen. Noter ned:

- apogee [m]
- maksimumshastighet
- tid til apogee [s]
- total flytid [s]



Figur 10: Viktige menyvalg i OpenRocket

Aktivitet 5: Databehandling

Behandling av data er en viktig del av enhver oppgave som involverer målinger. For å oppnå gode resultater og trekke nyttige konklusjoner, er det avgjørende å ha riktig informasjon om hva som blir målt og hva som er målet for oppdraget.

Datafilen som er lagret på SD-kortet etter rakettskuddet er avgjørende for å få innsikt i hva som skjedde når raketten var i lufta. Datafilen inneholder målinger av tid, temperatur, luftfuktighet, trykk, akselerasjon, rotasjon og magnetfelt. Det er også mulig å få ut orienteringen (*heading, pitch, roll*) til raketten igjennom lufta. Målet med denne oppgaven er å få analysert dette datasettet.

Det er ønskelig at all databehandling skal skje på PC-en, etter at målingene er gjort, og ikke med Arduinoen under selve rakettskuddet. Dette vil bidra til å minimere risikoen for at noe går galt, og minimere tiden Arduinoen på å gjøre målinger.

For å komme i gang med databehandlingen, er det første steget å sjekke om SD-kortet inneholder måledata. Dette kan gjøres ved å sette inn SD-kortet i en PC og sjekke filene som er lagret der. Hvis det ikke finnes måledata på kortet på grunn av tekniske problemer under oppdraget, vil oppgaven være å finne ut hva som gikk galt. Det er også mulig å bruke data fra tidligere skudd slik at databehandlingen kan gjennomføres.

```
# Eksempel: Importering av data fra SD kort til python
import pandas as pd

# Filbanen til CSV-filen som skal leses inn
filename = '/path/to/data.csv'

# Les inn CSV-filen og lagre dataene i en DataFrame
data = pd.read_csv(filename)

# Skriv ut DataFrame til konsollen
print(data)
```

For å håndtere dataene kan utdelt Python-kode brukes, f.eks. *Modellrakettdataleser.py*, men den må redigeres litt for å løse oppgaven. Det er også mulig å lage egen Python-kode eller bruke Excel hvis det er foretrukket. Uansett metode som brukes, vil behandling av data gi verdifull informasjon om hva som skjedde under rakettskuddet og hvordan raketten presterte i lufta.

Oppgave 1: Plott og les av data med Python

a) Plot høyden som funksjon av tid. Finn følgende størrelser fra høydemålingene:

- raketts apogee (høyeste punkt)
- hvor lang tid det gikk fra raketten tok av til den når apogee
- total flytid

Vurder hvordan målingene stemmer overens med simuleringer og baneberegninger.

b) Plot raketts totalakselerasjonen. Basert på akselerasjonen, redegjør for tidspunktene:

- når motoren brente
- når raketten var i glideflukt (i luften uten fallskjem)
- når raketts fallskjem kom ut
- når raketten traff bakken

Oppgave 2: Dataanalyse

a) Basert på målingene prøv å estimer gjennomsnittshastigheten til raketten når motoren brenner.

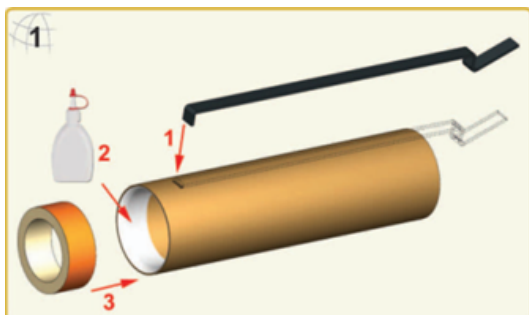
Hint: hastighet = strekning/tid.

b) Redegjør for hvordan høyden beregnes. Utforsk om alternative høydemodeller kan brukes.

c) Plott magnetfelt og/eller rotasjon, temperatur, luftfuktighet og redegjør for hva målingene viser.

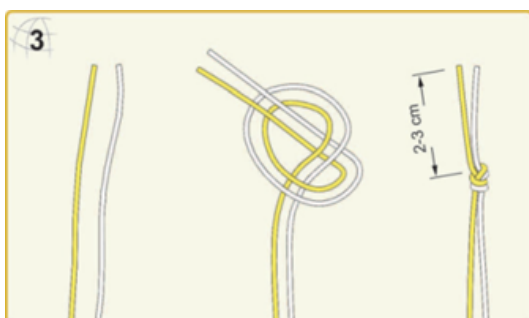
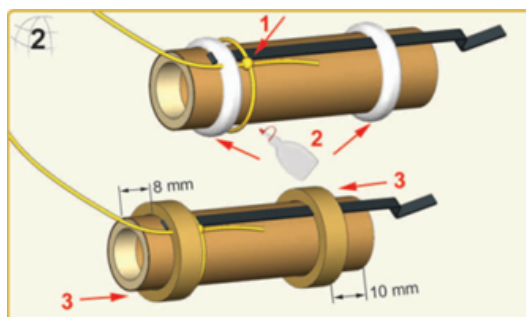
Instruksjoner: Bygging av modellrakett

Instruksjoner for bygging av ASE Custom 35mm rakett. Instruksjonene er basert på byggesettet Klima Sixpack Quick and Easy MAXI. Verktøy som trengs for bygging er lim, saks og sandpapir.



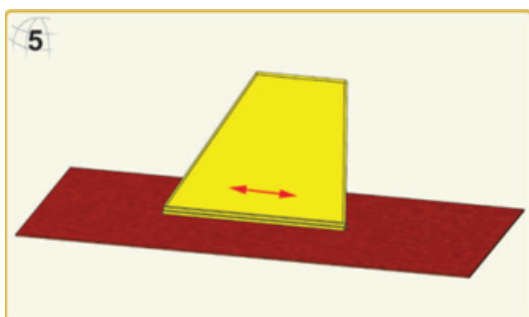
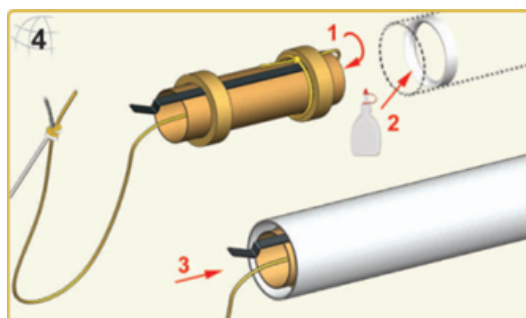
1. Fest metallstanga (motorfestet) med 90°-vinkelen inn i snittet i motorrøret. Lim deretter den minste ringen inni motorrøret slik at den er i kontakt med motorfestet på innsiden av røret.

2. Knyt den gule kevlartråden rundt motorrøret. Lim deretter på de to pappringene på utsiden av motorrøret. Merk at kevlartråden skal ligge på undersiden av den øverste pappringen slik som på figuren. Det kan derfor være nødvendig å skjære et lite hakk i ringen slik at den glir over tråden.

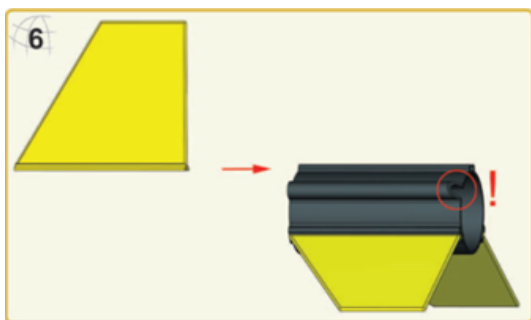


3. Knyt sammen den løse enden av kevlartråden og den elastiske tråden slik som vist på figuren. Sikre knuten med litt lim.

4. Tre kevlartråden og den elastiske tråden ut gjennom baksiden av motorrøret. Deretter limes motorrøret inn i rakettkroppen slik at enden på motorrøret er i nivå med enden på rakettkroppen. Når motorrøret er på plass mates kevlartråden inn gjennom motorrøret helt til den kommer ut på andre siden av rakettkroppen.

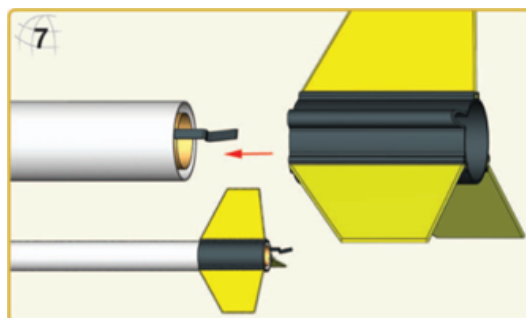


5. Puss innsiden av finnene med sandpapir. Finnene skal passe inn i slissenene på finneholderen.



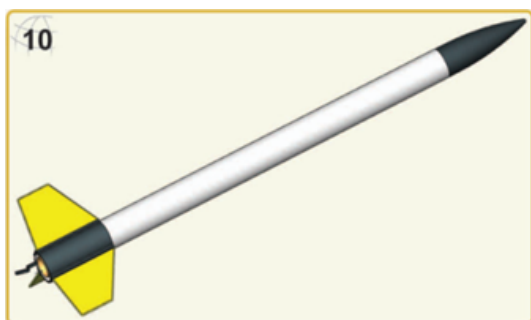
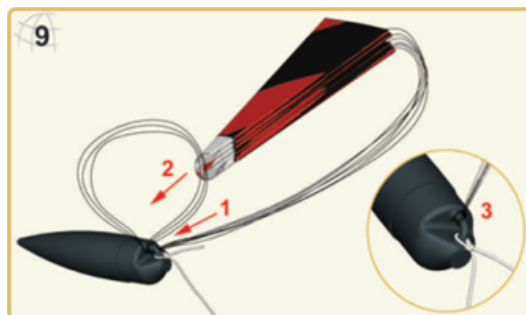
6. Skyv finnene på plass i finneholderen slik som vist på figuren. Det skal ikke være nødvendig å lime. Merk at den avrundede siden på den uthulede sylindren på siden av finneholderen skal vende nedover.

7. Skyv finneholderen på rakettkroppen slik at den er i flukt med bakenden på rakettkroppen.



8. Knyt den løse enden til den elastiske tråden godt fast i nyttelasten (neseskonen).

9. Sett sammen fallskjermen ved å følge instruksjonene som følger med fallskjermen. De løse løkkene fra fallskjermen tres sammen igjennom festet til nyttelasten der hvor den elastiske tråden er knytt fast. Deretter tres hele fallskjermen igjennom løkkene. Dra forsiktig til slik at løkkene strammer seg opp. Fallskjermen skal nå være festet.

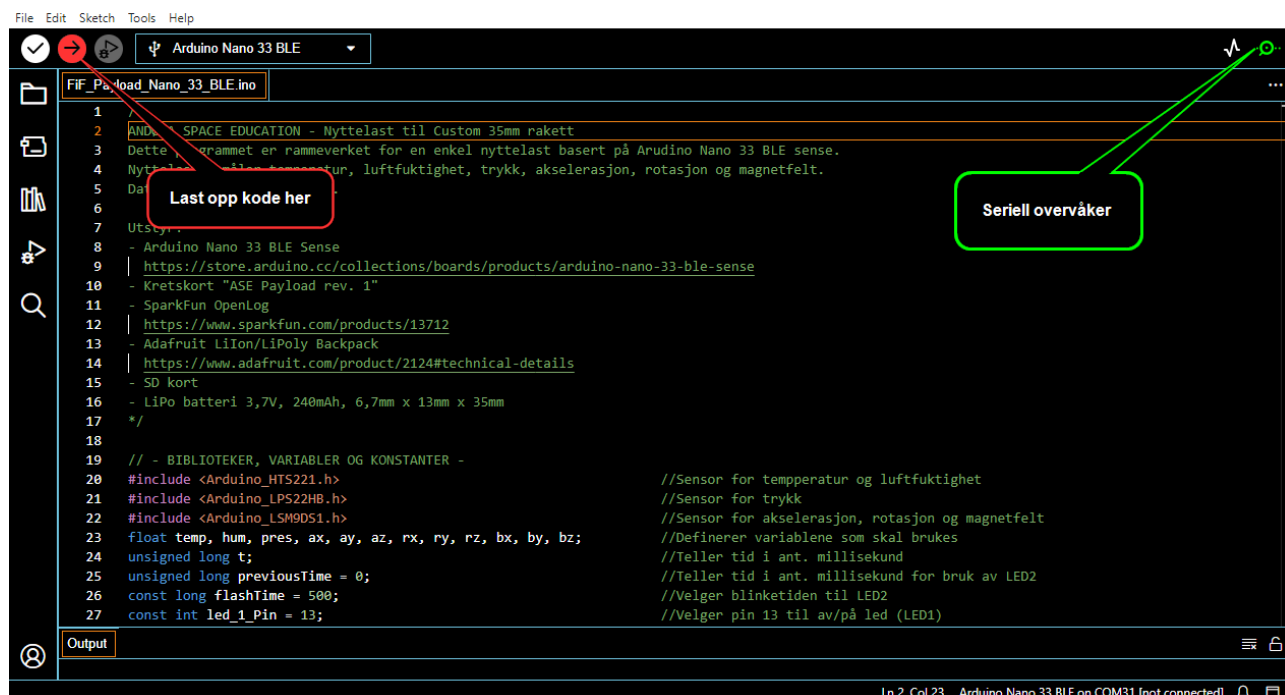


10. Wadding legges i rakettkroppen før fallskjermen legges i. Nyttelasten settes til slutt på. Den skal ikke limes! Raketten er nå ferdig. Den bør tørke i minst en time før oppskyting. Raketten kan dekores med farger eller andre effekter om det er ønskelig.

Instruksjoner: Nyttelast til modellrakett

En nyttelast er et fellesbegrep for en nyttig ting som er med en rakett under flukt. Det kan for eksempel være elektronikk som radiosendere, sensorer og datamaskiner eller i noen tilfeller mennesker. Nyttelasten er i vårt tilfelle en Arduino med sensorer, et batteri og et SD-kort. Aktiviteten går ut på å sette sammen, teste og lese data fra nyttelasten.

Arduino er en åpen kildekode program- og maskinvare produsent. De utvikler og produserer mikrokontrollerkort som kan brukes i en rekke applikasjoner. Kortene har sett med digitale og analoge innganger noe som gjør det mulig å koble til en rekke sensorer og annen maskinvare. Kortene er kompatible med programmeringsspråkene C og C++. ArduinoIDE er programvaren som oftest brukes for å kommunisere med mikrokontrollerne, og baserer seg på C++.



Skjermdump: Arduino IDE brukes for å redigere og laste opp kode til Arduinoen. Legg merke til knappene for Last opp"og Seriell overvåker"

Utstyr

- PC med Arduino IDE og USB kabel
- Arduino Nano 33 BLE Sense / Arduino Nano 33 BLE Sense Rev. 2
- ASE Payload kretskort
- Sparkfun OpenLog
- Adafruit LiIon/LiPoly Backpack
- Mikro SD kort
- LiPo batteri 3,7 V, 240 mAh, 6,7 mm x 13 mm x 35 mm
- 3D printet beholder
- Nesecone
- 2 x skruer M 2,5 x 35 mm og 33 mm
- 2 x muttere M 2,5
- 2 mm sekskantnøkkel

Oppgave 1: Last opp kode til nyttelasten

Oppgave 1 og 2 må fullføres før oppgave 3! Start Arduino IDE og åpne ett nytt tomt vidnu.

File → **New**

- a) Koble til Arduino Nano 33 BLE Sense via USB porten på PCen. Sjekk at Arduino IDE finner kortet ved å trykke på:

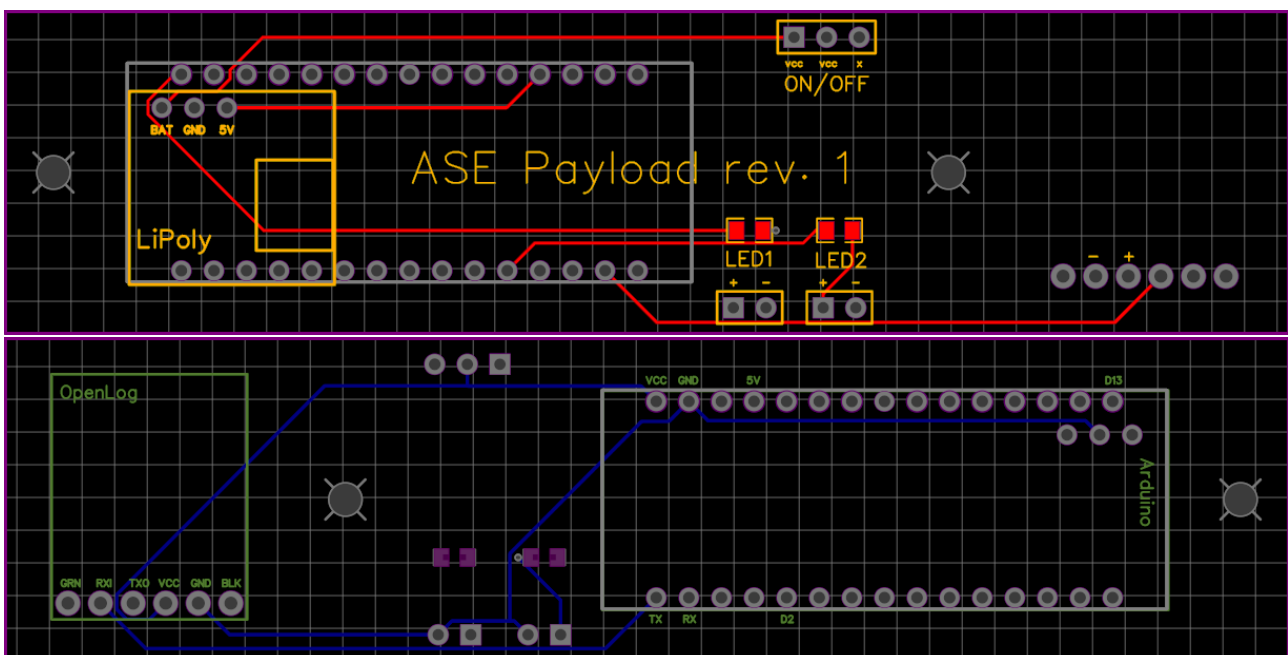
Tools → **Port**

Trykk på COM porten hvor kortet er koblet til. Hvis ikke kortet ligger i lista over porter må kortet legges til under:

Tools → **Board** → **Boards Manager** → Søk etter **Nano 33 BLE** og velg installer

Sjekk på nytt om kortet er koblet til ved å sjekke **Port**

- b) Lagre en lokal kopi av Arduino koden som hører til Arduinoen som skal brukes, for eksempel *FiF_Payload_Nano_33_BLE.ino*, PCen. Det er også mulig å skrive en egen kode til nyttelasten. Ta gjerne utgangspunkt i koden som er tilgjengelig.
- c) Åpne koden med Arduino IDE og se over koden og forklaringstekstene. Last deretter opp koden til Arduinoen (se skjermdumpen på forrige side). Det er ikke uvanlig at man må skifte port etter å ha trykket på **last opp**. Når koden er lastet opp sjekk at data fra sensorene skrives ved å sjekke seriell overvåkeren. Hvis data skrives er nyttelasten klar for testing i oppgave 2!



Figur 11: Plasing av komponenter på kretskort

Oppgave 2: Test

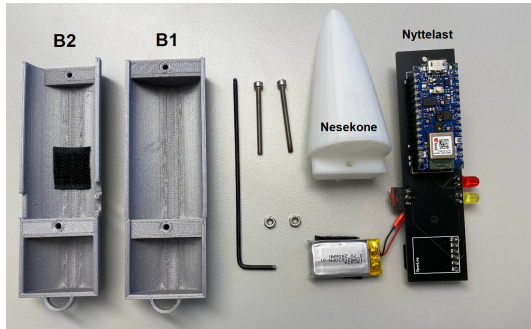
Koble Arduinoen fra PCen. Monter Arduinoen på kretskortet. Koble til et batteri og slå på nyttelasten med på-av knappen.

- Rød LED indikerer at nyttelasten har strøm
- Gul blinkende LED indikerer at data sendes over seriell port

- Lading av batteri kan gjøres ved å slå av nyttelasten, koble Arduino til PCen via USB samtidig som komponentene står på kretskortet og batteriet er koblet til. En rød LED på LiPoly laderen indikerer at batteriet lader, mens en blå LED vises når batteriet er oppladet

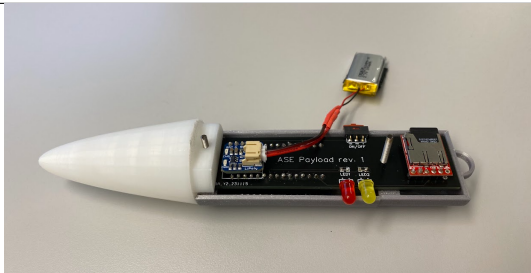
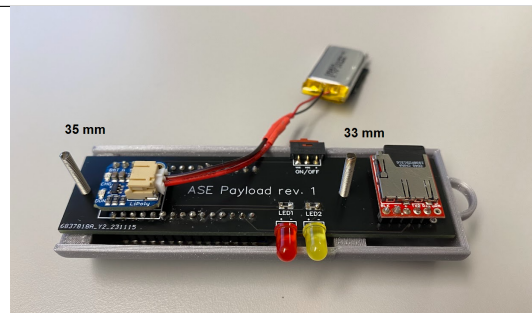
Nyttelasten fungerer hvis data skrives til SD kortet. Dette kan sjekkes ved ta SD-kortet ut å se om det ligger ny data på kortet.

Oppgave 3: Montering



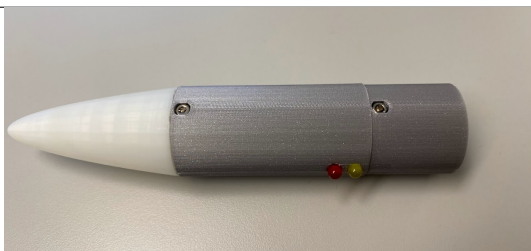
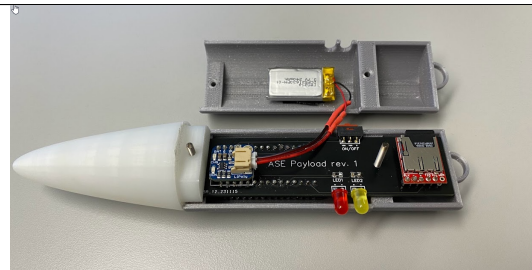
Start med å finne frem alle delene som trengs for monteringen av nyttelasten. Følg instruksjonene under for å montere nyttelasten i beholderen. Nyttelasten må være ferdig testet før monteringen starter!

Sett de to skruene i B1. Den lengste (35 mm) i hullet nærmest kanten, fra nå av kalt øvre kant. Den korteste (33 mm) i nederste hull. Legg nyttelasten med Arduino vendt ned i B1 slik at skruene treffer hullene på nyttelasten



Nesekonen settes på skruen ved øvre kant

Legg batteriet i B2 slik at borrelåsen holder det på plass. La ledningene være vendt ned



Til slutt settes B2 på de to skruene. Sett to muttere i hullene på del to slik at de treffer skruene. Stram **for-siktig** til med sekskantnøkkelen. Ikke stram for hard det kan skade beholderen. Sørg for at nyttelasten er avslått. Den skal ikke slås på før rett før oppskyting. Nyttelasten festes til fallskjermen som vist i steg 9 i instruksjonene for bygging.

Presentasjoner

Det skal holdes en kort presentasjon som en del av avslutningen på kurset. Presentasjonen skal gi et innblikk i rakettkampanjen dere har gjennomført. Dere velger selv hva presentasjonen skal inneholde og hvordan dere skal presentere innholdet. Lysbildefremvisning er mulig å bruke, men innholdet kan også presenteres på andre måter. Vær gjerne kreative.

Tips til presentasjon

- Presenter de viktigste resultatene, herunder:
 - tidslinje som viser tiden for rakettsens liftoff, brenntid, glidetid, fallskjemtid og totale flytid. Bruk gjerne figur 2 på side 4 som inspirasjon
 - simuleringer og baneberegninger
 - måledata/resultater fra raketteksperiment
- Presenter metode for innhenting av data, beregninger og simuleringer
- Sammenlign baneberegninger og/eller simuleringer med måledata
- Redegjør for viktige feilkilder
- Vis bilder/video av raketten og arbeidet som er gjort
- Hva har rollene innad i gruppa vært, og hvordan har dere håndtert å fordele arbeidsoppgavene?
- Hva har dere lært?

Referanser

- [1] T.S. Van Milligan and Inc Apogee Components. *Model Rocket Design and Construction: How to Create and Build Unique and Exciting Model Rockets That Work!* Apogee Components, Incorporated, 2000.
- [2] NASA Glenn Research Center. Beginner Guide to Aeronautics: Rocket Control. <https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/rocket-control/>, 2023. [Hentet online 06.12.2023].
- [3] NASA Glenn Research Center. Beginner Guide to Aeronautics: Model Solid Rocket Engine. <https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/model-solid-rocket-engine/>, 2023. [Hentet online 06.12.2023].
- [4] Sigbjørn Hervik. Eulers metode i Store norske leksikon på snl.no. https://snl.no/Eulers_metode, 2023. [Hentet online 06.12.2023].

Vedlegg

Vedlegg 1: Nedtellingsprosedyre

Ansvar og roller under oppskyting

Rolle	Forkortelse	Hvem?	Ansvar
Range safety officer	RSO		• sørger for at sjekklista i nedtellingsprosedyren blir fulgt, og krysser etterhvert som oppgaver blir utført • ansvarlig for å se hvor raketten lander. Krever at man holder øye med raketten gjennom hele flukten!
Launch control	LC		• ansvarlig for å sette raketten på rampa, koble tenningsystem og montere lunte • gjennomføre nedtelling høyt og tydelig fra 5 • trykke på avfyrringsknappen
Payload manager	PM		• sørge for at nyttelasten er koblet til strøm, skriver data og er sikkert plassert i raketten • ansvarlig for eventuell videodokumentasjon av rakettskuddet.
Instructor/teacher	INS		• tar med rampe, tenner, lunter og vernebriller til oppskytingssted • godkjenner rakett i henhold til sjekkliste i nedtellingsprosedyre
All stations	ALL	Hele gruppen	

Nedtellingsprosedyre

Tidspunkt relativt til oppskyting HH-MM-SS	Nr. #	Hvem utfører og hvem rapporteres det til		Kryss når utført	Ingen kritisk status	Beskrivelse av oppgave
					Nyttelast på intern strøm	
					Rakett på rampen	
					Tenning armert	
00-45-00	1	RSO	ALL			Sjekk værmeldingen for oppskytingssted ved planlagt tidspunkt for oppskyting
	2	RSO	ALL			Bekreft at simuleringer og baneberegninger for raketten er gjennomført og godkjent
	3	ALL	RSO			Les vedlegget «Sikker oppskyting av modellraketter» og gjør deg godt kjent med oppgavene i denne nedtellingsprosedyren
	4	INS	ALL			Raketten skal godkjennes av instruktør. Instruktør sjekker at: - o raketten er bygget i henhold til instruksjonene - o riktig motor montert - o nyttelast og rakettkropp kan separeres enkelt - o finnene er festet tilstrekkelig - o motorfestet er limt godt fast - o knutene mellom rakett og fallskjerm ser bra ut - o raketten inneholder wadding eller annen flammedemper for beskyttelse av fallskjerm
00-02-00	5	ALL	RSO			Ta på vernebriller
	6	RSO	ALL			Sjekk at alle tilskuere er på trygg avstand fra raketrampen, minimum 15 meter
	7	RSO	ALL			Sjekk at værforholdene er innenfor kriteriene for oppskyting
	8	LC	RSO			Sett raketten på rampen
	9	PM	RSO			Skru på nyttelasten og bekreft at lysindikator er aktivert
	10	LC	RSO			Koble tenningssystemet til lunta
	11	LC	RSO			Sett lunta inn i rakettmotoren
00-00-30	12	RSO	ALL			Sjekk at alle på gruppa er på trygg avstand fra raketrampen, minimum 5 meter
	13	PM	RSO			Bekreft at PM er klar for videodokumentasjon av rakettskuddet
	14	LC	RSO			Armer tenningssystemet
	15	LC	ALL			Tell ned fra T -5 høyt og tydelig
00-00-00	16	LC				Oppskyting
	17	RSO				Visuell sporing av raketten helt til den har landet
	18	RSO	ALL			Bekreft landing og husk landingssted for å gjøre henting/leting enklere
	19	LC	RSO			Desarmer tenningssystemet
	20	ALL	RSO			Legg vernebriller på plass
	21	ALL				Hent raketten når alle rakettene er skutt opp
	22	PM	RSO			Skru av nyttelast når raketten er funnet

Sikker oppskyting av modellraketter

Denne sjekklisten er utviklet av Andøya Space Education med utgangspunkt i Luftfartstilsynets forskrift om luftfartøy som ikke har fører ombord og The National Association of Rocketry (NAR) sine retningslinjer for sikker oppskyting av modellraketter. Denne veiledningen er laget for elever, lærere, vitensenterpedagoger og andre modellrakettentusiaster som en retningslinje for trygg bruk av modellraketter.

Generelle bestemmelser fra forskrift om luftfartøy som ikke har fører om bord

Modellraketter faller under definisjonen modellfly som vi finner under §4 Definisjoner og forkortelser i forskrift om luftfartøy som ikke har fører om bord. Derfor forholder vi oss til de generelle bestemmelsene i denne forskriften som gjelder modellfly.

«Modellfly: innretninger som ikke har fører om bord, som beveger seg i luften og hvor flygingen kun har rekreasjon, sport eller konkurranse som formål» – §4 Definisjoner og forkortelser, Forskrift om luftfartøy som ikke har fører om bord.

Ifølge §6 Begrensninger for flygingen skal all flyging med modellfly skje på en hensynsfull måte som ikke utsetter luftfartøy, personer, fugler, dyr eller eiendom for risiko for skade eller for øvrig er til sjenanse for allmennheten.

Modellflyet må til enhver tid være godt synlig for den som fører det slik at full kontroll over modellflyet kan opprettholdes. Flyging med modellfly kan bare skje i dagslysperioden og ikke:

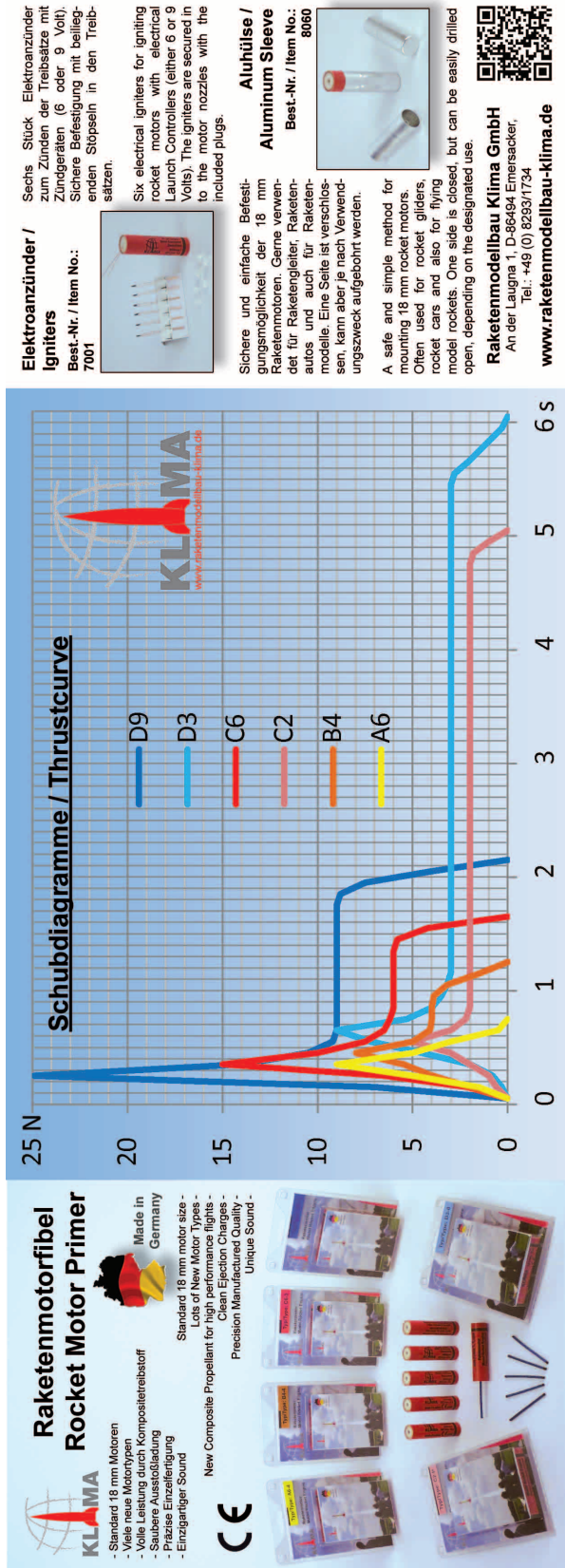
1. høyere enn 120 meter over bakken eller vannet
2. nærmere enn 150 meter fra personer, motorkjøretøy eller bygning som ikke er under fartøyførerens kontroll med unntak av under start og landing
3. nærmere enn 5 km fra lufthavner

Sjekkliste for oppskyting av modellraketter

1. Materialer: Modellrakettenes nesecone, kropp og finner skal bygges i lette materialer. Ikke metall eller andre tyngre materialer.
2. Motorer: Det skal kun brukes sertifiserte motorer fra seriøse leverandører. Les alltid medfølgende bruksanvisning nøye før bruk av rakettmotorer.
3. Antenning: Modellraketten skal alltid skytes opp med en elektrisk tenner og elektrisk lunte. Tenneren skal ha en sikringsfunksjon som kan slås av og på. Avfyringsknappen skal av seg selv gå tilbake i **av** posisjon etter at den er trykket inn.
4. Feiltenning: Hvis raketten ikke antennes skal sikring til tenneren slås på, eller batteriet til tenneren fjernes. Deretter skal en vente i minst 60 sekunder før man nærmer seg raketten.
5. Sikkerhet ved oppskyting: Enhver oppskyting skal ha egen nedtelling slik at alle tilskuere er oppmerksomme på oppskytingen. Alle tilskuere skal holde minst 15 m avstand. Den som trykker på tenneren skal bruke vernebriller, og ha en avstand på minst 5 m fra utskytingsplattformen.
6. Utskytingsrampe: Det skal brukes en dedikert rakettrampe, f.eks. en utskytningsplattform med startstang. Vinkelen til rakettrampen skal kunne justeres, og ved oppskyting skal vinkelen alltid være mellom 70° og 85° fra horisontalplanet, slik at raketten flyr omtrent rett opp. Det skal også brukes en varmedeflektor for å forhindre den varme gassen fra motoren i å treffe bakken. Ved bruk av startstang bør enden på stanga sikres mellom hver oppskyting for å forhindre øyeskader.
7. Størrelse: Rakettenes startvekt skal ikke overgå 250 gram. Største totalimpuls på motoren skal ikke overgå 20 Ns (motorkategori D).
8. Sikkerhet under flyging: Modellraketter skal kun skytes i klar sikt. Raketter skal ikke skytes ved lavt skydekke eller i nærheten av fugler, fly eller andre objekter i lufta. Ved bruk av nyttelast skal ikke denne være brennbar, eksplosiv eller utgjøre noen annen fare for raketten eller omgivelsene.

9. Egnet sted for oppskyting: Raketter skal kun skytes utendørs i et stort åpent område med minst 200×200 m klaring, og hvor alle personer som oppholder seg innenfor dette området er informert om aktiviteten. Sjekk alltid værmeldingen før planlagt oppskyting av modellraketter. Rakettoppskytinger skal ikke skje ved middelvind som overgår 7 m/s. Oppskyting skal heller ikke skje i områder som er eksponert for skogbrann.
10. Fallskjerm: Enhver modellrakett skal ha fallskjerm slik at den kan lande sikkert. Det skal også alltid brukes et flammebestandig materiale inni raketten for å beskytte fallskjermens funksjon når denne skytes ut, f.eks. wadding.
11. Sikkerhet ved henting: Raketter skal ikke hentes hvis de lander på utilgjengelige steder, f.eks. høyt oppe i trær eller i strømledninger.

Vedlegg 2: Rakettmotorspesifikasjoner (Klima)



Best-Nr. / Item No.		Type	Gesamtimpuls / Total Impulse	Schub / Thrust	Schubdauer / Burning Period	Verzögerung / Delay	Gesamtgewicht / Total Weight	Pulvergewicht / Powder Weight	Max. Startgewicht / Max. Lift-Off Wgt.	Verwendung / Usage
0164	A6-4	2,5 Ns	6 N	ca. 0,6 s	ca. 4 s	14,5 g	3,5 g	70 g		Single Stage
0240	B4-0	5,0 Ns	4 N	ca. 1,3 s	0 s	15,8 g	5,0 g	90 g		Final Stage in multi-stage rocket
0244	B4-4	5,0 Ns	4 N	ca. 1,3 s	ca. 4 s	16,8 g	5,8 g	90 g		Final Stage in multi-stage rocket
0320	C2-0	10,0 Ns	2 N	ca. 5,0 s	0 s	22,3 g	11,5 g	150 g		Booster Stage
0329	C2-P	10,0 Ns	2 N	ca. 5,0 s	verschlossen / plugged	22,5 g	11,0 g	150 g		Booster Stage
0360	C6-0	10,0 Ns	6 N	ca. 1,6 s	0 s	18,7 g	8,9 g	160 g		Booster Stage
0363	C6-3	10,0 Ns	6 N	ca. 1,6 s	ca. 3 s	20,5 g	9,5 g	160 g		Booster Stage
0365	C6-5	10,0 Ns	6 N	ca. 1,6 s	ca. 5 s	21,2 g	10,2 g	150 g		Booster Stage
0367	C6-7	10,0 Ns	6 N	ca. 1,6 s	ca. 7 s	22,0 g	11,0 g	140 g		Booster Stage
0369	C6-P	10,0 Ns	6 N	ca. 1,6 s	verschlossen / plugged	19,9 g	8,4 g	160 g		Booster Stage
0430	D3-0	19,0 Ns	3 N	ca. 6,0 s	0 s	28,0 g	17,2 g	200 g		Booster Stage
0439	D3-P	19,0 Ns	3 N	ca. 6,0 s	verschlossen / plugged	27,7 g	16,7 g	200 g		Booster Stage
0490	D9-0	20,0 Ns	9 N	ca. 2,1 s	0 s	26,3 g	15,5 g	300 g		Booster Stage
0493	D9-3	20,0 Ns	9 N	ca. 2,1 s	ca. 3 s	27,1 g	16,1 g	300 g		Booster Stage
0495	D9-5	20,0 Ns	9 N	ca. 2,1 s	ca. 5 s	27,6 g	16,6 g	250 g		Booster Stage
0497	D9-7	20,0 Ns	9 N	ca. 2,1 s	ca. 7 s	28,2 g	17,2 g	200 g		Booster Stage
0499	D9-P	20,0 Ns	9 N	ca. 2,1 s	verschlossen / plugged	26,5 g	15,0 g	300 g		Booster Stage

Die Motoren werden in Packungen zu 6 Stück mit beiliegenden Zündschnüren geliefert. Alternativ sind Elektroanzünder erhältlich. Die Motorlänge beträgt 70 mm und der Durchmesser ist 17,8 mm. Each motor type comes in a pack of six, including safety fuses for igniting. Electric igniters are separately available. Motor dimensions are 17,8 mm in diameter and 70 mm in length.